

Module 04 — Content-Based Recommendation (NLP)

src/content_based_nlp.py • 313 lines

In one sentence

Recommends products by their text, which is how new products get recommended at all.

1. What this module does

Every other model in this project works out what to recommend by looking at **who bought what**. That approach has one fatal blind spot: a product listed this morning has no buyers, so it is invisible.

This module solves that. It reads each product's written description, tags, category and brand, converts that text into numbers using TF-IDF, and compares products by how similar their text is. A brand-new product has all of that from the moment it is listed - so it can be recommended on day one.

It also builds a 'taste profile' for each customer by averaging the text of the products they liked, and can explain *which words* make two products similar.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
TF-IDF	Score words by how telling they are	Converting descriptions to numbers
Term frequency	How often a word appears in one document	The TF part
Inverse document frequency	Rare words matter more than common ones	The IDF part
Stop words	Ignore 'the', 'and', 'is'	stop_words='english'
N-grams	Treat 'stainless steel' as one term	ngram_range=(1, 2)
Cosine similarity	How close two vectors point	Comparing product vectors
Vector space model	Documents as points in space	The whole approach
Centroid / user profile	Average of what you liked	build_user_content_profile()
Item cold start	Recommending something with no sales	The main purpose of this module
Sparse matrices	Store only the non-zero values	The TF-IDF output

3. How it works, step by step

1. **Build one text document per product** by joining the description, category, subcategory, brand and tags. Hyphens in tags are removed so 'noise-cancelling' also matches the separate words.
2. **Fit TF-IDF** over all 2,500 documents, producing a 2,500 x 5,000 matrix of word importance scores.
3. **Compute cosine similarity** between every pair of products, giving a 2,500 x 2,500 similarity grid.
4. **For similar-item recommendations**, just read the row for that product and take the highest scores.

5. For customer recommendations, average the text vectors of the products they liked to get a taste profile, then compare that profile against everything.

6. For a brand-new customer, fall back to their declared category and their segment's typical price band.

4. Key functions

Function	What it does
build_item_text()	Joins description, category, brand and tags into one string
create_tfidf_features()	Fits the vectoriser and builds the matrix
create_content_similarity_matrix()	Product-to-product similarity grid
recommend_similar_items()	Top-N most similar products
build_user_content_profile()	Average of the products a customer liked
recommend_for_existing_user()	Ranks the catalogue against that profile
get_cold_start_items()	Finds products with almost no history
recommend_for_new_user_by_profile()	First recommendations from registration data
top_shared_terms()	Which exact words make two products similar

5. Inputs, outputs and how to run

	Detail
Inputs	items.csv, interactions.csv
Outputs	tfidf_vectorizer.pkl, tfidf_matrix.pkl, content_similarity.pkl
Key numbers	5,000-word vocabulary • 75.4% catalogue coverage
Runtime	About 20 seconds
Run with	python src/content_based_nlp.py

TF-IDF settings and why each one is set

Setting	Value	Why
stop_words	'english'	'the' and 'and' say nothing about a product
ngram_range	(1, 2)	keeps 'stainless steel' as one meaningful term
min_df	2	drops words appearing in only one product - usually a model code
sublinear_tf	True	repeating a word ten times is not ten times as important
max_features	5000	caps the vocabulary so the matrix stays fast

6. Key code

src/content_based_nlp.py — create_tfidf_features()

```

68 | def create_tfidf_features(items_df: pd.DataFrame, max_features: int = MAX_TFIDF_FEATURES):
69 |     """Fit the TF-IDF vectoriser over the catalogue."""
70 |     items_with_text = build_item_text(items_df)
71 |
72 |     # min_df=2 drops single-product terms (model codes); bigrams keep
73 |     # multi-word attributes like "stainless steel" as one feature.
74 |     vectorizer = TfidfVectorizer(
75 |         stop_words="english",
76 |         max_features=max_features,
77 |         ngram_range=(1, 2),
78 |         min_df=2,
79 |         sublinear_tf=True,
80 |     )
81 |
82 |     tfidf_matrix = vectorizer.fit_transform(items_with_text["content_text"])
83 |
84 |     save_pickle(vectorizer, "tfidf_vectorizer.pkl")
85 |     save_pickle(tfidf_matrix, "tfidf_matrix.pkl")
86 |
87 |     items_with_text[["item_id", "title", "category", "description", "content_text"]].to_csv(
88 |         os.path.join(PROCESSED_DATA_PATH, "item_text_metadata.csv"), index=False
89 |     )
90 |
91 |     return items_with_text, vectorizer, tfidf_matrix

```

Four settings do the real work here - see the table below for what each one is for.

src/content_based_nlp.py — recommend_for_new_user_by_profile()

```

217 | def recommend_for_new_user_by_profile(preferred_category: str, items_df: pd.DataFrame,
218 |                                     user_segment: str = None, n_top: int = 10) -> pd.DataFrame:
219 |     """First-touch recommendations from registration data alone."""
220 |     candidates = items_df.loc[
221 |         items_df["category"].str.lower() == str(preferred_category).lower()
222 |     ].copy()
223 |
224 |     if candidates.empty:
225 |         candidates = items_df.copy()
226 |
227 |     # Price-band matching stops a Budget Shopper being shown the most
228 |     # expensive item in their category.
229 |     if user_segment in SEGMENT_PRICE_PERCENTILE and "price_percentile" in candidates.columns:
230 |         target = SEGMENT_PRICE_PERCENTILE[user_segment]
231 |         candidates["price_fit"] = 1.0 - (candidates["price_percentile"] - target).abs()
232 |         quality = candidates["base_quality"] if "base_quality" in candidates.columns else 0.5
233 |         candidates["cold_start_score"] = 0.6 * candidates["price_fit"] + 0.4 * quality
234 |     else:
235 |         candidates["cold_start_score"] = (
236 |             candidates["base_quality"] if "base_quality" in candidates.columns else 0.5
237 |         )
238 |
239 |     columns = [c for c in ["item_id", "title", "category", "subcategory", "brand",
240 |                           "price", "cold_start_score"] if c in candidates.columns]
241 |
242 |     return candidates.sort_values("cold_start_score", ascending=False)[columns].head(n_top)

```

This is what stops a budget shopper being shown the most expensive item in their favourite category.

7. Things to remember

This is the answer to item cold start

Cold-start product #3 in this dataset has **zero** interactions. Collaborative filtering cannot score it at all. The content model matches it to other Home & Kitchen storage products at 0.54 similarity, driven by shared brand and the terms 'dishwasher safe' and 'storage'. That product can be shown to customers from day one and start building the history the other models need.

8. Likely questions

Q. What does TF-IDF actually do?

A. It turns text into numbers. A word scores highly for a product if it appears often in that product's description (term frequency) but rarely across the whole catalogue (inverse document frequency). So 'waterproof' is informative, while 'product' is not.

Q. Why is this model the weakest on accuracy but still included?

A. Its NDCG@10 is only 0.0116, but its catalogue coverage is 75.4% - more than three times any other model. It is the only component that can reach the long tail, so it earns a small weight in the hybrid for reach rather than accuracy.

Q. How do you explain why two products are similar?

A. `top_shared_terms()` multiplies the two TF-IDF vectors together element by element. The highest products are the words contributing most to the similarity score, so 'similarity 0.54' becomes 'both are dishwasher-safe Claypeak storage'.